Rakuten Product Classification — Visualizations 3+

This notebook contains the **visualizations from the newer audit notebook starting with Figure 3**, but adapted so it can run inside the existing `liora-capstone` GitHub project.

The data is loaded at the beginning using the project's existing helper:

```
df = load_merged_data()
```

Figures 1 and 2 from the newer notebook are intentionally omitted.

1. Imports

```
import math

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
from IPython.display import Markdown, display

from src.data_utils import load_merged_data, load_image

RANDOM_STATE = 42

pd.set_option("display.max_columns", 50)
pd.set_option("display.max_colwidth", 120)

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams.update({
    "figure.figsize": (12, 6),
    "axes.titleweight": "bold",
    "axes.spines.top": False,
    "axes.spines.right": False,
})

PALETTE = {
    "primary": "#BF0000",
    "secondary": "#7A0019",
    "accent": "#E3A008",
    "neutral": "#64748B",
    "light": "#E2E8F0",
}

print("Environment loaded successfully.")
```

```
Environment loaded successfully.
```

2. Load Data

```
df = load_merged_data()

print("Dataset shape:", df.shape)
display(df.head(5))
```

```
Dataset shape: (84916, 5)
```

	designation	description	productid	imageid	prdtypecode
0	Olivia: Personalisiertes Notizbuch / 150 Seiten / Punktraster / Ca Din A5 / Rosen-Design	NaN	3804725264	1263597046	10
1	Journal Des Arts (Le) N° 133 Du 28/09/2001 - L'art Et Son Marche Salon D'art Asiatique A Paris - Jacques Barrere - F...	NaN	436067568	1008141237	2280
2	Grand Stylet Ergonomique Bleu Gamepad Nintendo Wii U - Speedlink Pilot Style	PILOT STYLE Touch Pen de marque Speedlink est 1 stylet ergonomique pour GamePad Nintendo Wii U. Pour un confort ...	201115110	938777978	50
3	Peluche Donald - Europe - Disneyland 2000 (Marionnette À Doigt)	NaN	50418756	457047496	1280
4	La Guerre Des Tuques	Luc a des idées de grandeur. Il veut organiser un jeu de guerre de boules de neige et s'arranger pour en &eci...	278535884	1077757786	2705

3. Audit Feature Construction

The following analytical features are required by Figures 3 onward.
The original dataframe `df` remains unchanged.

```python
def feature_frame(data: pd.DataFrame) -> pd.DataFrame:
    result = data.copy()

    designation = result["designation"].fillna("").astype(str)
    description = result["description"].fillna("").astype(str)

    result["has_description"] = (
        result["description"].notna()
        & description.str.strip().ne("")
    )

    result["designation_chars"] = designation.str.len()
    result["designation_words"] = designation.str.findall(r"\b\w+\b").str.len()

    result["description_chars"] = description.str.len()
    result["description_words"] = description.str.findall(r"\b\w+\b").str.len()

    result["total_text_chars"] = (
        result["designation_chars"]
        + result["description_chars"]
    )

    normalized = (
        designation
        .str.normalize("NFKD")
        .str.encode("ascii", errors="ignore")
        .str.decode("ascii")
        .str.lower()
        .str.replace(r"\s+", " ", regex=True)
        .str.replace(r"[^a-z0-9 ]", "", regex=True)
        .str.strip()
    )

    result["designation_normalized"] = normalized

    result["contains_html"] = description.str.contains(
        r"<[^>]+>",
        regex=True,
        na=False,
    )

    result["contains_url"] = (
        designation + " " + description
    ).str.contains(
        r"https?://|www\.",
        regex=True,
        case=False,
        na=False,
    )

    result["contains_mojibake"] = (
        designation + " " + description
    ).str.contains(
        r"Ã.|Â.|â€|",
        regex=True,
        na=False,
    )

    return result
```

```
train_audit = feature_frame(df)

display(
    train_audit[
        [
            "designation",
            "has_description",
            "designation_chars",
            "description_chars",
            "total_text_chars",
        ]
    ].head()
)
```

	designation	has_description	designation_chars	description_chars	total_text_chars
0	Olivia: Personalisiertes Notizbuch / 150 Seiten / Punktraster / Ca Din A5 / Rosen-Design	False	88	0	88
1	Journal Des Arts (Le) N° 133 Du 28/09/2001 - L'art Et Son Marche Salon D'art Asiatique A Paris - Jacques Barrere - F...	False	206	0	206
2	Grand Stylet Ergonomique Bleu Gamepad Nintendo Wii U - Speedlink Pilot Style	True	76	760	836
3	Peluche Donald - Europe - Disneyland 2000 (Marionnette À Doigt)	False	63	0	63
4	La Guerre Des Tuques	True	20	213	233

```python
def insight(
    title: str,
    observation: str,
    evidence: str,
    business: str,
) -> None:
    display(
        Markdown(
            f"### {title}\n\n"
            f"**Observation.** {observation}\n\n"
            f"**Quantitative evidence.** {evidence}\n\n"
            f"**Business implication.** {business}"
        )
    )


def cramers_v(contingency: pd.DataFrame) -> float:
    chi2 = stats.chi2_contingency(
        contingency,
        correction=False,
    )[0]

    n = contingency.to_numpy().sum()
    rows, columns = contingency.shape

    phi2 = chi2 / n

    phi2_corrected = max(
        0,
        phi2
        - ((columns - 1) * (rows - 1))
        / max(n - 1, 1),
    )

    rows_corrected = (
        rows
        - ((rows - 1) ** 2)
        / max(n - 1, 1)
    )

    columns_corrected = (
        columns
        - ((columns - 1) ** 2)
        / max(n - 1, 1)
    )

    denominator = min(
        columns_corrected - 1,
        rows_corrected - 1,
    )

    return (
        math.sqrt(phi2_corrected / denominator)
        if denominator > 0
        else 0.0
    )
```

4. Figure 3 — Description Availability by Class

If missing descriptions are associated with the target, description availability itself may contain category-related information.

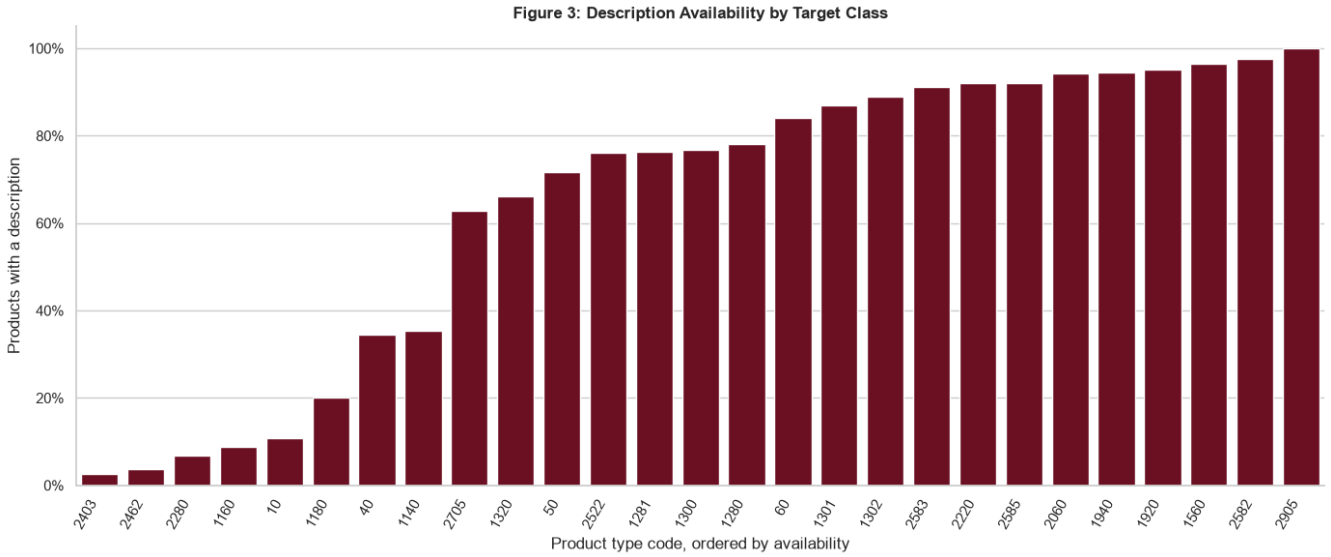
```
description_by_class = (
    train_audit
    .groupby("prdtypecode", as_index=False)
    .agg(
        products=("has_description", "size"),
        description_available=("has_description", "mean"),
    )
    .sort_values("description_available")
)

plt.figure(figsize=(14, 6))

ax = sns.barplot(
    data=description_by_class,
    x="prdtypecode",
    y="description_available",
    order=description_by_class["prdtypecode"],
    color=PALETTE["secondary"],
)

ax.set_title(
    "Figure 3: Description Availability by Target Class"
)
ax.set_xlabel(
    "Product type code, ordered by availability"
)
ax.set_ylabel(
    "Products with a description"
)
ax.yaxis.set_major_formatter(
    lambda value, _: f"{value:.0%}"
)

plt.xticks(
    rotation=60,
    ha="right",
)
plt.tight_layout()
plt.show()
```




```
description_class_table = pd.crosstab(
    train_audit["prdtypecode"],
    train_audit["has_description"],
)

chi2, p_value, _, _ = stats.chi2_contingency(
    description_class_table
)

v_description_class = cramers_v(
    description_class_table
)

availability_range = (
    description_by_class["description_available"].min(),
    description_by_class["description_available"].max(),
)

insight(
    "Finding 3 — missing descriptions are category-dependent",
    (
        f"Class-level description availability ranges from "
        f"{availability_range[0]:.1%} to "
        f"{availability_range[1]:.1%}."
    ),
    (
        f"χ²={chi2:,.1f}, "
        f"p={p_value:.3g}, "
        f"Cramér's V={v_description_class:.3f}."
    ),
    (
        "Description presence may improve classification, "
        "but it can also reflect seller behaviour rather than product semantics."
    ),
)
```

Finding 3 — missing descriptions are category-dependent

Observation. Class-level description availability ranges from 2.6% to 100.0%.

Quantitative evidence. χ^2=43,757.8, p=0, Cramér's V=0.718.

Business implication. Description presence may improve classification, but it can also reflect seller behaviour rather than product semantics.

5. Figure 4 — Designation Length by Class

```
class_order = (
    train_audit
    .groupby("prdtypecode")["designation_chars"]
    .median()
    .sort_values()
    .index
)

plt.figure(figsize=(15, 7))

ax = sns.boxplot(
    data=train_audit,
    x="prdtypecode",
    y="designation_chars",
    order=class_order,
    color="#D97706",
    showfliers=False,
)

ax.set_title(
    "Figure 4: Designation Length by Target Class"
)

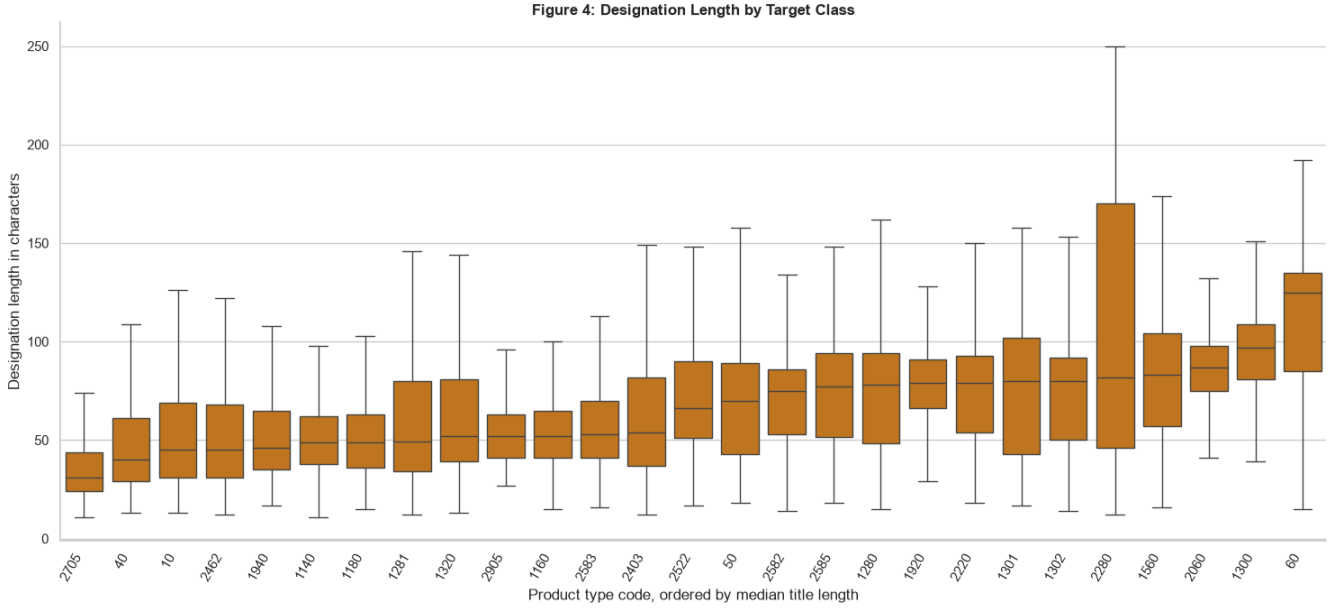
ax.set_xlabel(
    "Product type code, ordered by median title length"
)

ax.set_ylabel(
    "Designation length in characters"
)

plt.xticks(
    rotation=60,
    ha="right"
)

plt.tight_layout()
plt.show()
```

Figure 4: Designation Length by Target Class



```
groups = [
    group["designation_chars"].to_numpy()
    for _, group
    in train_audit.groupby(
        "prdtypecode",
        sort=False,
    )
]

h_stat, p_value = stats.kruskal(
    *groups
)

n = len(train_audit)
k = len(groups)

epsilon_squared = max(
    0.0,
    (h_stat - k + 1)
    / (n - k),
)

median_range = (
    train_audit
    .groupby("prdtypecode")["designation_chars"]
    .median()
    .agg(["min", "max"])
)

insight(
    "Finding 4 — title length contains target-related structure",
    (
        f"Median designation length by class ranges "
        f"from {median_range['min']:.0f} to "
        f"{median_range['max']:.0f} characters."
    ),
    (
        f"Kruskal–Wallis H={h_stat:,.1f}, "
        f"p={p_value:.3g}; "
        f"epsilon-squared={epsilon_squared:.3f}."
    ),
    (
        "Text models should consider both word and character information. "
        "For neural models, sequence length should be chosen from the observed
distribution."
    ),
)
```

Finding 4 — title length contains target-related structure

Observation. Median designation length by class ranges from 31 to 125 characters.

Quantitative evidence. Kruskal–Wallis H=19,025.3, p=0; epsilon-squared=0.224.

Business implication. Text models should consider both word and character information. For neural models, sequence length should be chosen from the observed distribution.

6. Figure 5 — Relationship Between Designation and Description Length

```
with_description = train_audit.loc[
    train_audit["has_description"],
    [
        "designation_chars",
        "description_chars",
    ],
].copy()

plot_sample = with_description.sample(
    min(
        30_000,
        len(with_description),
    ),
    random_state=RANDOM_STATE,
)

plot_sample = plot_sample.assign(
    designation_log=np.log10(
        1 + plot_sample["designation_chars"]
    ),
    description_log=np.log10(
        1 + plot_sample["description_chars"]
    ),
)

plt.figure(figsize=(10, 7))

hexbin = plt.hexbin(
    plot_sample["designation_log"],
    plot_sample["description_log"],
    gridsize=55,
    mincnt=1,
    bins="log",
    cmap="magma",
)

plt.colorbar(
    hexbin,
    label="log10 product count",
)

from matplotlib.ticker import FuncFormatter

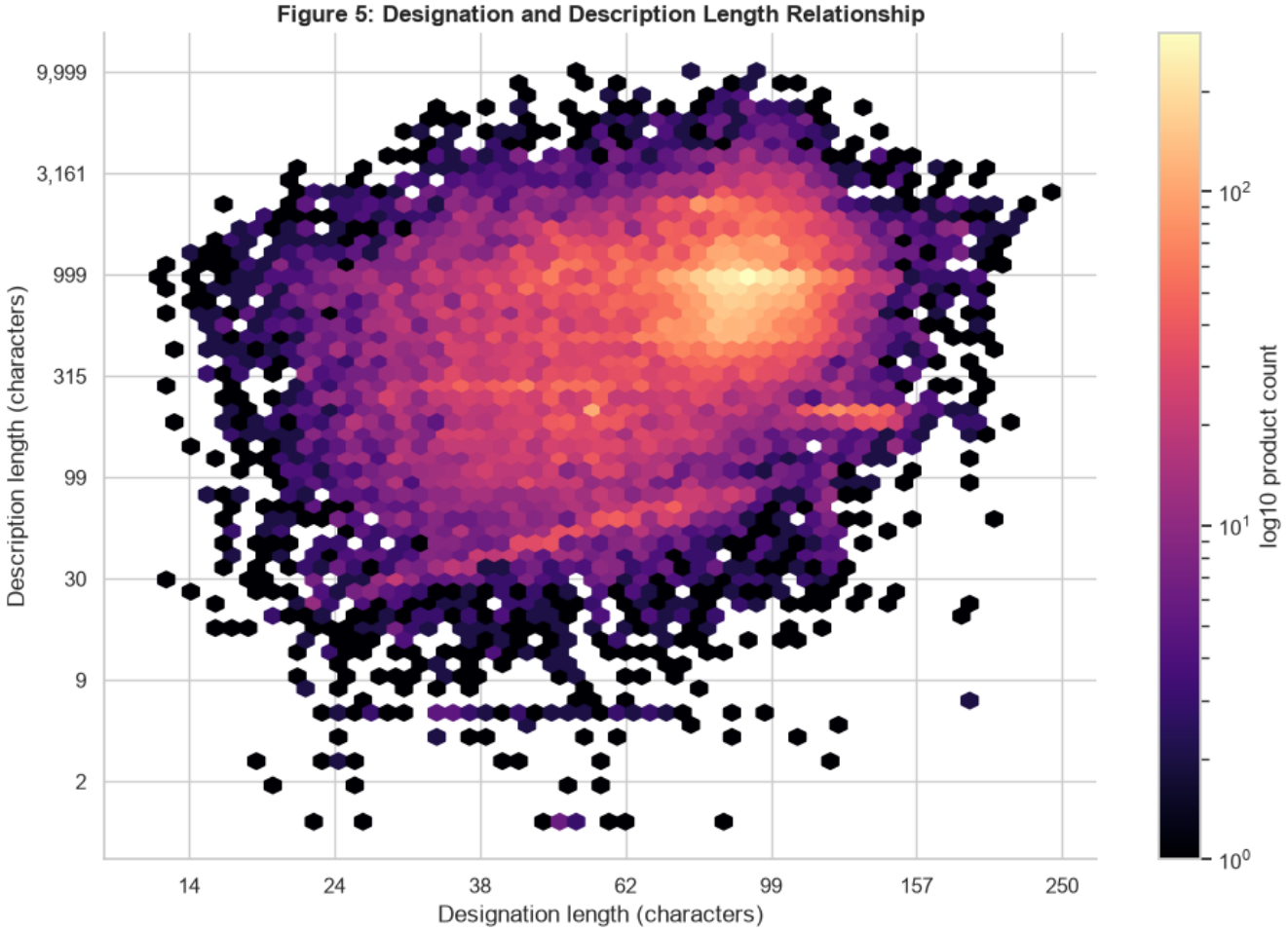
plt.title(
    "Figure 5: Designation and Description Length Relationship"
)

plt.xlabel("Designation length (characters)")
plt.ylabel("Description length (characters)")

# Convert log10(1 + chars) back to character counts for axis labels
formatter = FuncFormatter(
    lambda x, _: f"{int(10**x - 1):,}"
)

plt.gca().xaxis.set_major_formatter(formatter)
plt.gca().yaxis.set_major_formatter(formatter)
```

```
plt.tight_layout()
plt.show()
```



“Descriptions are much longer than designations, while their lengths are only moderately correlated. Therefore, we should test whether combining both text fields improves classification.”

8. Figure 7 — Duplicate Designations and Conflicting Labels

```
title_stats = (
    train_audit
    .groupby(
        "designation_normalized",
        dropna=False,
    )
    .agg(
        rows=(
            "designation_normalized",
            "size",
        ),
        classes=(
            "prdtypecode",
            "nunique",
        ),
    )
)

duplicate_titles = (
    title_stats.loc[
        title_stats["rows"] > 1
    ]
)

duplicate_rows = int(
    duplicate_titles["rows"]
    .sum()
)

conflicting_groups = (
    duplicate_titles.loc[
        duplicate_titles["classes"] > 1
    ]
)

conflicting_rows = int(
    conflicting_groups["rows"]
    .sum()
)

duplicate_by_class = (
    train_audit
    .assign(
        is_duplicate=
        train_audit[
            "designation_normalized"
        ].duplicated(
            keep=False
        )
    )
    .groupby(
        "prdtypecode",
        as_index=False,
    )
    .agg(
        products=(
            "prdtypecode",
            "size",
        ),
```

```
        duplicate_rate=(
            "is_duplicate",
            "mean",
        ),
    )
    .sort_values(
        "duplicate_rate",
        ascending=False,
    )
)

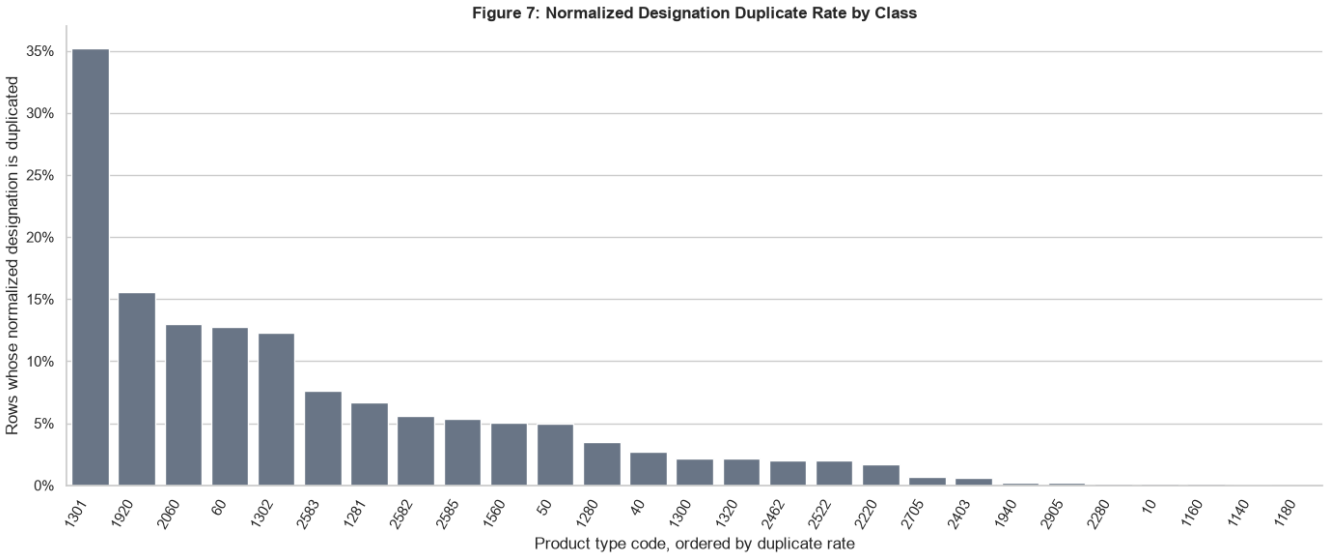
plt.figure(figsize=(14, 6))

ax = sns.barplot(
    data=duplicate_by_class,
    x="prdtypecode",
    y="duplicate_rate",
    order=duplicate_by_class[
        "prdtypecode"
    ],
    color=PALETTE["neutral"],
)

ax.set_title(
    "Figure 7: Normalized Designation Duplicate Rate by Class"
)
ax.set_xlabel(
    "Product type code, ordered by duplicate rate"
)
ax.set_ylabel(
    "Rows whose normalized designation is duplicated"
)

ax.yaxis.set_major_formatter(
    lambda value, _: f"{value:.0%}"
)

plt.xticks(
    rotation=60,
    ha="right",
)
plt.tight_layout()
plt.show()
```




```
insight(
    "Finding 7 — repeated designations and conflicting labels",
    (
        f"{duplicate_rows:,} training rows "
        f"({duplicate_rows / len(train_audit):.1%}) "
        f"belong to a repeated normalized designation. "
        f"{conflicting_rows:,} rows belong to duplicate-title groups "

        f"containing more than one target class."
    ),
    (
        f"There are {len(duplicate_titles):,} repeated-title groups and "
        f"{len(conflicting_groups):,} conflicting-label groups."
    ),
    (
        "Review conflicting groups before deciding whether they represent "
        "true duplicate records or legitimate catalogue entries."
    ),
)
```

Finding 7 — repeated designations and conflicting labels

Observation. 4,172 training rows (4.9%) belong to a repeated normalized designation. 497 rows belong to duplicate-title groups containing more than one target class.

Quantitative evidence. There are 1,463 repeated-title groups and 160 conflicting-label groups.

Business implication. Review conflicting groups before deciding whether they represent true duplicate records or legitimate catalogue entries.

4.172 Zeilen = 4,9 % des gesamten Trainingsdatensatzes haben einen normalisierten Titel, der mindestens zweimal vorkommt. Von diesen Duplikaten gehören 497 Zeilen zu problematischen Gruppen, bei denen derselbe Titel unterschiedliche Klassenlabels besitzt.

Titel Klasse

"Samsung Galaxy S20" 10 "Samsung Galaxy S20" 10 ← normales Duplikat

"Harry Potter DVD" 40 "Harry Potter DVD" 50 ← conflicting duplicate

A **duplicate group** contains all rows that have the same normalized designation.

Example:

Row	Normalized designation	Class
1	iphone 12 black	10
2	iphone 12 black	10
3	iphone 12 black	10
4	harry potter dvd	40
5	harry potter dvd	50
6	sony headphones	30
7	sony headphones	30

This gives:

- **7 duplicate rows**
- **3 duplicate groups**
  - `iphone 12 black`
  - `harry potter dvd`
  - `sony headphones`

Only one group is conflicting:

- `harry potter dvd` → appears with class 40 and class 50

So:

- Duplicate rows = 7
- Duplicate groups = 3
- Conflicting groups = 1

9. Text-Quality Flags

```
quality_columns = [
    "contains_html",
    "contains_url",
    "contains_mojibake",
]

quality_rates = (
    train_audit[
        quality_columns
    ]
    .mean()
    .to_frame("train")
)

quality_counts = pd.DataFrame({
    "flag": quality_columns,
    "train_rows": [
        int(
            train_audit[column]
            .sum()
        )
        for column
        in quality_columns
    ],
})

display(
    quality_rates.style.format(
        "{:.2%}"
    )
)

display(
    quality_counts
)
```

	train
contains_html	18.42%
contains_url	0.24%
contains_mojibake	1.08%

	flag	train_rows
0	contains_html	15645
1	contains_url	203
2	contains_mojibake	918

10. Training Image Duplicate Audit — File Hashes

This section checks **exact image duplicates in the training catalog only**. Two images are duplicates when their complete file bytes have the same SHA-256 hash.

The earlier audit showed that file and decoded-pixel hashes produced exactly the same duplicate groups in this dataset. Only the simpler file hash is therefore retained. No perceptual hash is used, the cache is stored under `data/processed/`, and the raw images are not modified.

```python
import hashlib
import json
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

from src.data_utils import get_data_paths, get_image_path


def fingerprint_image(image_path: str) -> dict:
    """Return a portable SHA-256 fingerprint of the complete file bytes."""
    path = Path(image_path)
    result = {"image_filename": path.name, "error": None}
    try:
        stamp = path.stat()
        raw = path.read_bytes()
        result.update(
            size=stamp.st_size,
            mtime_ns=stamp.st_mtime_ns,
            file_sha256=hashlib.sha256(raw).hexdigest(),
        )
    except (OSError, ValueError) as error:
        result["error"] = f"{type(error).__name__}: {error}"
    return result


def audit_image_catalog(catalog: pd.DataFrame, cache_path: Path) -> pd.DataFrame:
    """Return one audit row per catalog row and cache hashes by filename."""
    cache_version = "training-file-sha256-v2"
    cached = {}
    if cache_path.exists():
        try:
            stored = json.loads(cache_path.read_text(encoding="utf-8"))
            for key, values in stored.get("images", {}).items():
                filename = values.get("image_filename") or Path(key).name
                cached[filename] = values
        except (OSError, ValueError):
            print("Hash cache could not be read; images will be rescanned.")

    records, pending = {}, []
    for image_path in catalog["image_path"].drop_duplicates():
        filename = Path(image_path).name
        previous = cached.get(filename, {})
        try:
            stamp = Path(image_path).stat()
            unchanged = (
                previous.get("size") == stamp.st_size
                and previous.get("mtime_ns") == stamp.st_mtime_ns
                and previous.get("file_sha256") is not None
                and previous.get("error") is None
            )
        except OSError:
            unchanged = False
        if unchanged:
            records[filename] = {
                "image_filename": filename,
                "error": None,
                "size": previous["size"],
                "mtime_ns": previous["mtime_ns"],
                "file_sha256": previous["file_sha256"],
```

```
            }
        else:
            pending.append(image_path)

    print(f"Images reused from cache: {len(records):,}; to scan: {len(pending):,}")
    with ThreadPoolExecutor(max_workers=4) as executor:
        for scanned, result in enumerate(executor.map(fingerprint_image, pending),
1):
            records[result["image_filename"]] = result
            if scanned % 10_000 == 0 or scanned == len(pending):
                print(f"Hashed {scanned:,}/{len(pending):,} images")

    cache_path.parent.mkdir(parents=True, exist_ok=True)
    temporary = cache_path.with_suffix(".tmp")
    temporary.write_text(
        json.dumps({"version": cache_version, "images": records}),
        encoding="utf-8",
    )
    temporary.replace(cache_path)

    fingerprints = pd.DataFrame.from_records(list(records.values()))
    for column in ["file_sha256", "error"]:
        if column not in fingerprints:
            fingerprints[column] = None
    return catalog.merge(fingerprints, on="image_filename", how="left",
validate="many_to_one")


data_paths = get_data_paths()
image_catalog = df[["imageid", "productid", "prdtypecode"]].copy()
image_catalog["source_row"] = image_catalog.index
image_catalog["image_path"] = [
    get_image_path(int(row.imageid), int(row.productid), split="image_train")
    for row in image_catalog.itertuples(index=False)
]
image_catalog["image_filename"] = image_catalog["image_path"].map(lambda path:
Path(path).name)

image_hash_cache = Path(data_paths["x_train"]).parent.parent / "processed" /
"training_image_duplicate_hashes.json"
image_duplicate_audit = audit_image_catalog(image_catalog, image_hash_cache)
image_failures =
image_duplicate_audit.loc[image_duplicate_audit["file_sha256"].isna()]

print(f"Successfully hashed images: {len(image_catalog) - len(image_failures):,}/
{len(image_catalog):,}")
if not image_failures.empty:
    display(image_failures[["source_row", "image_path", "error"]].head(10))

assert len(image_duplicate_audit) == len(image_catalog)
assert image_duplicate_audit["file_sha256"].notna().all(), "At least one image has
no file hash."
```

```
Images reused from cache: 0; to scan: 84916
Hashed 10000/84916 images
Hashed 20000/84916 images
Hashed 30000/84916 images
Hashed 40000/84916 images
Hashed 50000/84916 images
Hashed 60000/84916 images
```

```
Hashed 70000/84916 images
Hashed 80000/84916 images
Hashed 84916/84916 images
Successfully hashed images: 84916/84916
```

```
def collect_duplicate_groups(audit: pd.DataFrame, hash_column: str) ->
pd.DataFrame:
    """Return exact duplicate groups from the training image catalog."""
    group_sizes = (
        audit.dropna(subset=[hash_column])
        .groupby(hash_column)
        .size()
        .rename("rows")
        .reset_index()
    )
    return (
        group_sizes.loc[group_sizes["rows"].gt(1)]
        .sort_values(["rows", hash_column], ascending=[False, True])
        .reset_index(drop=True)
    )


file_duplicate_groups = collect_duplicate_groups(image_duplicate_audit,
"file_sha256")

image_duplicate_summary = pd.Series({
    "file_duplicate_groups": len(file_duplicate_groups),
    "rows_in_duplicate_groups": int(file_duplicate_groups["rows"].sum()),
}, name="count").to_frame()
display(image_duplicate_summary)

# List the ten largest training duplicate groups.
duplicate_group_preview = (
    file_duplicate_groups
    .assign(hash_prefix=lambda table: table["file_sha256"].str[:12])
    .head(10)
    .reset_index(drop=True)
    [["hash_prefix", "rows"]]
)
display(duplicate_group_preview)

# File-hash group sizes and label conflicts within the training set.
file_duplicate_details = (
    image_duplicate_audit.groupby("file_sha256")
    .agg(
        products_per_group=("file_sha256", "size"),
        class_count=("prdtypecode", "nunique"),
    )
    .loc[lambda table: table["products_per_group"].gt(1)]
)

file_group_size_distribution = (
    file_duplicate_details.groupby("products_per_group")
    .size()
    .rename("duplicate_groups")
    .reset_index()
)
file_group_size_distribution["products_in_groups"] = (
    file_group_size_distribution["products_per_group"]
    * file_group_size_distribution["duplicate_groups"]
)
display(file_group_size_distribution)

conflicting_file_groups = file_duplicate_details.loc[
```

```
    file_duplicate_details["class_count"].gt(1)
]
file_conflict_summary = pd.Series({
    "file_duplicate_groups": len(file_duplicate_details),
    "conflicting_file_groups": len(conflicting_file_groups),
    "rows_in_conflicting_groups": int(
        conflicting_file_groups["products_per_group"].sum()
    ),
}, name="count").to_frame()
display(file_conflict_summary)
```

	count
file_duplicate_groups	3264
rows_in_duplicate_groups	8956

	hash_prefix	rows
0	1a07ac586847	47
1	818f6c09f96c	26
2	e811419be694	26
3	346527e3aa29	22
4	8420074fdaf9	21
5	2423a6c4aedc	20
6	afa2a66a219b	20
7	ce682062fdd8	20
8	a8f7abb17656	19
9	5f5c00e5cc04	18

	products_per_group	duplicate_groups	products_in_groups
0	2	2303	4606
1	3	515	1545
2	4	187	748
3	5	80	400
4	6	41	246
5	7	39	273
6	8	35	280
7	9	16	144
8	10	12	120
9	11	4	44
10	12	3	36
11	13	7	91
12	14	3	42
13	15	5	75
14	16	2	32
15	17	1	17
16	18	2	36
17	19	1	19
18	20	3	60
19	21	1	21
20	22	1	22
21	26	2	52
22	47	1	47

	count
file_duplicate_groups	3264
conflicting_file_groups	479
rows_in_conflicting_groups	1074

Reading the results

`file_duplicate_groups` counts distinct file hashes that occur more than once. `rows_in_duplicate_groups` counts all training rows belonging to those groups. The preview lists the ten largest exact-file duplicate groups.

The group-size table shows how many exact file-hash groups contain 2, 3, 4 or more products. A **label conflict** means that the same exact file occurs with more than one `prdtypecode` in the training data. It is a review candidate, not automatically a labelling error.

11. Preprocessing Decisions Supported by Figures 3+

- Keep `has_description` as a candidate feature.
- Compare designation-only, description-only and combined-text models.
- Use word and character representations for text.
- Treat exact file duplicates as an audit finding; do not remove rows automatically.
- Keep image paths and loading centralized through `src.data_utils`.
- The portable training-image file-hash cache is stored under `data/processed/`.